RallyNIM Enterprise Product Requirements Document (PRD)

Part I – Product Vision & Strategy

Document Version: 1.0
Project: RallyNIM
Author: Precious Adebisi
Target Platform: Nimiq Pay Mini App
Frontend: React + TypeScript + Vite + TailwindCSS
Backend: Node.js + Express + TypeScript
Database: MongoDB Atlas
Wallet Integration: Nimiq Mini App SDK
Development Network: Nimiq Testnet
Production Network: Nimiq Mainnet

---

1. Executive Summary

Product Overview

RallyNIM is a next-generation Event Engagement Platform built exclusively for the Nimiq Pay Mini App ecosystem.

Rather than functioning as a traditional cryptocurrency airdrop tool, RallyNIM transforms passive events into interactive experiences by rewarding attendees in real time using NIM.

The platform enables organisers to create structured reward campaigns consisting of multiple engagement stages such as:

- Event Check-in
- Workshop Attendance
- Speaker Challenges
- Live Quizzes
- Networking Missions
- Treasure Hunts
- Sponsor Booth Visits
- Merchant Cashback
- Community Tasks

Participants complete activities and receive instant NIM rewards directly inside Nimiq Pay.

Every reward is transparent, verifiable and tied to an actual wallet interaction.

Vision Statement

> **To become the default engagement infrastructure for communities, conferences, universities, merchants and Web3 events by making NIM the universal incentive layer for participation.**

Instead of asking:

> "How do we distribute rewards?"

Organisers should ask:

> "How do we make our event engaging?"

RallyNIM provides that answer.

Mission

Build the most delightful reward platform inside the Nimiq ecosystem while significantly increasing:

- NIM transactions
- Wallet activity
- Daily active users
- Community participation
- Merchant adoption
- Event engagement

Product Philosophy

RallyNIM follows five design principles.

1. Reward Action

Users should earn rewards for doing meaningful things.

Not simply existing.

---

2. Zero Learning Curve

A first-time user should understand the application within **30 seconds**.

No manuals.

No tutorials.

---

3. Mobile First

Every interaction should feel native inside Nimiq Pay.

Desktop is secondary.

---

4. Every Click Matters

Every action should either

- increase engagement
- distribute value
- create excitement

No unnecessary screens.

---

5. Production over Prototype

The application should feel like something already used by thousands of people.

Not a hackathon demo.

---

2. Vision

Long-term Vision

Become the "Stripe for Event Rewards."

Whenever an organiser wants to incentivise attendance or participation, the first solution they think of should be RallyNIM.

Within five years the platform should support:

- Universities
- Conferences
- Music festivals
- Merchants
- NGOs
- DAOs
- Hackathons
- Gaming tournaments

- Loyalty programmes
- Community campaigns

across the Nimiq ecosystem.

Future Vision

Imagine attending ETHGlobal.

Instead of paper tickets...

Instead of manually claiming rewards...

Instead of waiting weeks for prizes...

You simply open Nimiq Pay.

Every achievement instantly unlocks NIM.

Your Event Passport updates.

Your leaderboard changes.

Your attendance badge appears.

Everything happens in seconds.

---

3. Problem Statement

Current Problems

Modern events struggle with engagement.

Organisers commonly face the following issues.

Problem 1

People check in...

...and disappear.

Attendance ≠ Participation.

Problem 2

Reward distribution is painful.

Organisers manually:

- collect wallet addresses
- export spreadsheets
- verify winners
- send payments individually

This wastes hours.

Problem 3

Sponsors receive little measurable engagement.

They don't know

- who visited
- who interacted
- who completed tasks

Problem 4

Community events have no incentive layer.

Most meetups rely on goodwill instead of rewarding participation.

Problem 5

Merchants cannot easily reward customers with NIM.

Cashback systems are expensive.

Existing loyalty systems require separate infrastructure.

Problem 6

Hackathons struggle to motivate participants throughout the event.

Most rewards happen only at the end.

This reduces continuous engagement.

Existing Solutions

Current solutions include

- Google Forms
- Paper tickets
- Manual transfers
- QR attendance systems
- Generic payment links
- Spreadsheet tracking

These solve attendance.

They do **not** solve engagement.

---

4. Market Opportunity

Primary Market

Web3 Events

Examples

- ETHGlobal
- Devcon
- Consensus
- Token2049
- Local meetups

Pain Point

Rewarding hundreds of attendees.

Universities

Examples

- Orientation
- Tech Clubs
- Blockchain Communities

- Workshops
- Career Fairs

Opportunity

Gamify participation.

Hackathons

Participants earn rewards for

- checking in
- submitting projects
- attending workshops
- completing sponsor tasks
- winning prizes

Conferences

Instead of giving everyone a gift bag...

Reward engagement.

Merchants

Examples

Coffee shops

Restaurants

Retail stores

Use NIM cashback.

Community Organisations

Examples

- DAOs
- NGOs
- Local communities

Reward volunteers.

Secondary Market

Gaming tournaments

Music festivals

Charity events

Sports competitions

Trade fairs

Market Trend

Consumer applications are increasingly using gamification, instant rewards, and digital wallets to drive engagement. At the same time, Web3 communities are looking for practical, everyday use cases beyond speculation.

RallyNIM sits at the intersection of these trends by turning NIM into a real-time engagement and incentive mechanism for both organisers and participants.

---

5. User Personas

Persona 1

Event Organiser

Age

24–45

Goals

- Reward attendees
- Increase participation
- Measure engagement
- Reduce administrative work

Pain Points

- Manual payments

- Low engagement
- Spreadsheet chaos

Needs

One-click campaign creation.

Persona 2

University Community Lead

Goals

Increase student participation.

Reward members.

Track attendance.

Needs

Reusable event templates.

Persona 3

Attendee

Goals

Earn rewards.

Participate.

Have fun.

Collect badges.

Needs

Fast onboarding.

Instant payouts.

No complicated setup.

Persona 4

Sponsor

Goals

Measure booth traffic.

Increase engagement.

Collect analytics.

Needs

Sponsor-specific campaigns.

Persona 5

Merchant

Goals

Reward customers.

Drive repeat visits.

Offer cashback.

Needs

Simple QR campaigns.

---

6. Success Metrics (KPIs)

Product Metrics

- Total campaigns created
- Active campaigns
- Campaign completion rate
- Average participants per campaign
- Campaign reuse rate

User Metrics

- Daily Active Wallets (DAW)
- Weekly Active Wallets (WAW)
- Monthly Active Wallets (MAW)
- Wallet retention (Day 1, Day 7, Day 30)
- Average rewards claimed per user
- Average sessions per week

Ecosystem Metrics

- Total NIM distributed
- Number of NIM transactions initiated
- Total transaction volume
- Average reward value
- New Nimiq Pay users referred

Engagement Metrics

- Average event completion rate
- Quiz participation
- Networking missions completed
- Treasure hunt completion
- Sponsor interactions
- Merchant redemptions

Competition Success Metrics

To maximise the judging score, RallyNIM will target:

- **Design & UX:** Users complete onboarding and claim a reward in under 60 seconds with a polished, mobile-first interface.
- **Functionality:** Stable wallet integration, reliable reward distribution, comprehensive error handling, and smooth performance.
- **Usefulness & Originality:** Campaigns that solve real organiser problems while encouraging repeat usage through reusable templates and engagement mechanics.
- **Marketing & Distribution:** Hundreds of unique wallet interactions, public development updates, demo videos, and active participation in the builder community.
- **NIM Usage:** Every campaign requires funding and every reward is distributed in NIM, making the token central to the product rather than an optional feature.

---

7. Competitive Analysis

Product	Strength	Weakness	Why RallyNIM Wins
Google Forms	Simple attendance	No rewards	Adds real incentives
Eventbrite	Event management	No engagement layer	Live NIM rewards
POAP	Digital collectibles	No financial incentives	Rewards + collectibles
Generic QR Attendance Apps	Check-ins	No gamification	Multi-stage campaigns
Loyalty Apps	Cashback	Merchant-specific	Supports events, communities and merchants
Manual Wallet Transfers	Flexible	Slow and error-prone	Automated, structured distribution

Unique Differentiators

1. Built specifically for the Nimiq Pay Mini App ecosystem.
2. Wallet-native onboarding with no account creation.
3. Multi-stage engagement campaigns rather than one-off payouts.
4. Smart Campaign Templates for rapid event setup.
5. Event Passport that records participation history and achievements.
6. Real-time analytics for organisers.
7. Instant NIM rewards that reinforce practical use of the Nimiq ecosystem.

---

8. Value Proposition

For Event Organisers

Create interactive reward campaigns in minutes, automate NIM distribution, and measure participation with live analytics instead of spreadsheets.

For Attendees

Complete activities, earn instant NIM, collect event badges, build an Event Passport, and enjoy a fun, rewarding event experience.

For Sponsors

Launch branded challenges, incentivise booth visits, and gain measurable engagement data instead of relying on foot traffic estimates.

For Merchants

Reward purchases with instant NIM cashback and encourage repeat visits through simple, wallet-native campaigns.

For the Nimiq Ecosystem

Every campaign increases wallet interactions, on-chain transactions, and practical use of NIM. By making rewards an integral part of real-world events and commerce, RallyNIM expands the utility and visibility of the Nimiq Pay ecosystem beyond traditional cryptocurrency use cases.

Smart Campaign Templates (Strategic Preview)

One of RallyNIM' defining features will be **Smart Campaign Templates**, allowing organisers to launch polished campaigns in under a minute without designing reward flows from scratch.

Planned templates include:

- 🎓 University Orientation
- 💻 Hackathon Journey
- 🎤 Conference Engagement
- 🛍️ Merchant Cashback
- 🏆 Tournament Rewards
- 🎮 Treasure Hunt
- 🤝 Community Meetup
- 🗳️ Governance & Voting Incentives
- 🎁 Product Launch Campaign
- 🌍 NGO & Volunteer Programme

Each template will come with preconfigured stages, reward logic, timing, and suggested NIM allocations while remaining fully customisable. This feature is intended to maximise usability, repeat value, and onboarding speed—key factors in the competition's judging criteria.

Part II – Product Requirements

9. Functional Requirements

9.1 Overview

RallyNIM consists of six core systems working together:

1. Authentication System
2. Campaign Management System
3. Reward Engine
4. Claim Engine
5. Analytics Engine
6. Event Passport System

Each system must operate independently while sharing a unified event model.

10. Functional Modules

Module A — Authentication

Description

Authentication is entirely wallet-based.

There are **no email/password accounts**.

Users authenticate using the Nimiq Mini App SDK by connecting their wallet and signing a nonce-based message. The backend verifies the signature and issues a short-lived JWT plus a refresh token.

Functional Requirements

FR-A1

Users can connect a Nimiq wallet.

FR-A2

Users authenticate by signing a message.

FR-A3

Backend verifies signature.

FR-A4

Generate JWT.

FR-A5

Store user profile.

FR-A6

Reconnect automatically on future visits.

FR-A7

Logout clears session.

Acceptance Criteria

✓ Wallet connected

✓ JWT generated

✓ User redirected

Module B — Campaign Management

Campaigns are the heart of RallyNIM.

Every event belongs to one campaign.

A campaign contains:

- Information
- Stages
- Rewards
- Rules
- Analytics

Campaign Properties

Campaign Name

Description

Location

Banner

Category

Organiser

Status

Reward Pool

Participants

Start Time

End Time

Visibility

Maximum Participants

Status

Draft

Scheduled

Live

Paused

Completed

Archived

Cancelled

Functional Requirements

Create Campaign

Edit Campaign

Duplicate Campaign

Pause Campaign

Resume Campaign

Delete Campaign

Publish Campaign

Archive Campaign

Module C — Campaign Stages

A campaign is divided into multiple stages.

Example:

Check-in

↓

Workshop

↓

Quiz

↓

Networking

↓

Closing Ceremony

Each stage has its own:

Reward

Trigger

Rules

Timer

Claim Method

Stage Properties

Stage Name

Description

Reward Amount

Maximum Claims

Availability Window

Trigger Type

Verification Method

Sponsor

Difficulty

Stage Status

Locked

Upcoming

Active

Completed

Expired

Module D — Reward System

Every reward belongs to one stage.

Rewards may be:

Fixed

Random

Leaderboard

Milestone

Lottery

Referral

Cashback

Achievement

Reward Properties

Reward ID

Campaign ID

Stage ID

Reward Type

Reward Amount

Budget

Remaining Budget

Maximum Winners

Module E — Event Passport

Every user has a permanent Event Passport.

It records:

Events attended

Rewards earned

Badges

Achievements

Total NIM earned

Campaign history

Participation streak

Leaderboard rankings

Future versions may allow users to export or share their passport as a profile.

Module F — Analytics

Analytics are available in real time.

Metrics include:

Live participants

Claims

Wallets

Reward distribution

Average completion

Peak activity

Popular stages

Top campaigns

Repeat users

---

11. User Stories

Organiser Stories

Story O-01

As an organiser

I want to create a campaign

So that I can reward participants.

Story O-02

As an organiser

I want to fund a reward pool with NIM

So rewards can be distributed during the event.

Story O-03

As an organiser

I want to duplicate previous campaigns

So I don't start from scratch.

Story O-04

As an organiser

I want to monitor live analytics

So I understand engagement.

Story O-05

As an organiser

I want to pause a campaign

If something unexpected happens.

Story O-06

As an organiser

I want to export campaign reports

For sponsors and stakeholders.

Participant Stories

Story P-01

As a participant

I want to claim rewards quickly

Without complicated registration.

Story P-02

As a participant

I want to view my reward history.

Story P-03

As a participant

I want to see upcoming campaign stages.

Story P-04

As a participant

I want my achievements saved.

Story P-05

As a participant

I want to share my rewards.

Story P-06

As a participant

I want to build attendance streaks.

Sponsor Stories

Create branded campaigns

Track booth engagement

View analytics

Reward visitors

Download reports

Merchant Stories

Create cashback campaigns

Verify purchases

Reward customers

View customer retention

---

12. Complete User Flows

Flow 1 — First-Time User

Open Nimiq Mini App

↓

Welcome

↓

Connect Wallet

↓

Sign Message

↓

Create User

↓

Dashboard

↓

Browse Campaigns

Target completion:

Under 30 seconds.

Flow 2 — Create Campaign

Dashboard

↓

Create Campaign

↓

Choose Template

↓

Edit Details

↓

Configure Stages

↓

Fund Reward Pool

↓

Preview

↓

Publish

↓

Campaign Live

Flow 3 — Join Campaign

Open Campaign

↓

View Details

↓

Join

↓

Wallet Verified

↓

Current Stage

↓

Claim Available

Flow 4 — Reward Claim

Complete Task

↓

Verification

↓

Eligible?

↓

Yes

↓

Reward Engine

↓

Wallet Confirmation

↓

Transaction

↓

Success Screen

↓

Passport Updated

Flow 5 — Campaign Completion

Last Stage

↓

Claim

↓

Campaign Finished

↓

Leaderboard

↓

Achievements

↓

Share Card

Flow 6 — Merchant Cashback

Purchase

↓

Scan QR

↓

Receipt Verified

↓

Reward Created

↓

Wallet Confirms

↓

Cashback Sent

---

13. Information Architecture

Home

├── Campaigns

│ ├── Live

│ ├── Upcoming

│ ├── Completed

│

├── My Rewards

│

├── Event Passport

│

├── Leaderboards

│

├── Notifications

│

├── Profile

│

├── Organiser Dashboard

│ ├── Campaigns

│ ├── Analytics

│ ├── Templates

│ ├── Reports

│

└── Settings

---

14. Smart Campaign Templates

One of RallyNIM' biggest differentiators.

Instead of creating campaigns manually...

Organisers choose a template.

Everything is automatically generated.

Template 1

🎓 University Orientation

Stages

- Check-in
- Department Booth
- Society Booth
- Quiz
- Feedback

Default Budget

100 NIM

Template 2

💻 Hackathon

Stages

Check-in

↓

Team Formation

↓

Workshop Attendance

↓

Project Submission

↓

Demo Day

↓

Prize Distribution

Template 3

🎤 Conference

Stages

Registration

↓

Opening Keynote

↓

Sponsor Booth

↓

Workshop

↓

Networking

↓

Closing

Template 4

🛍️ Merchant Cashback

Stages

Purchase

↓

Receipt Verification

↓

Cashback

↓

Referral Bonus

↓

Loyalty Milestone

Template 5

🎮 Treasure Hunt

Stages

Find QR

↓

Find QR

↓

Answer Puzzle

↓

Hidden Location

↓

Final Reward

Template 6

🤝 Community Meetup

Stages

Arrival

↓

Icebreaker

↓

Group Photo

↓

Feedback

↓

Lucky Draw

Template 7

🏆 Tournament

Stages

Registration

↓

Round 1

↓

Semi Final

↓

Final

↓

Winner Reward

Template 8

🌍 Volunteer Campaign

Stages

Check In

↓

Task Assignment

↓

Completion

↓

Verification

↓

Reward

Template Features

Every template includes:

Recommended budget

Suggested rewards

Suggested timings

Default stage order

Recommended anti-cheat settings

Analytics dashboard

Branding

Editable workflow

Organisers can also save custom templates for future use.

---

15. Reward Engine

The Reward Engine determines:

Who receives rewards.

How much.

When.

Why.

It is the core business logic of the application.

Reward Types

Fixed Reward

Every eligible participant receives the same amount.

Example:

2 NIM

Random Reward

User receives a random amount within a configured range.

Example:

1–5 NIM

Leaderboard Reward

Distributed based on ranking.

Example:

1st → 25 NIM

2nd → 15 NIM

3rd → 10 NIM

Milestone Reward

Unlock after completing multiple stages.

Attendance Reward

Given after verified attendance.

Referral Reward

Both inviter and invitee receive rewards.

Sponsor Reward

Sponsor-specific tasks.

Secret Code Reward

Requires a valid code.

QR Reward

Requires QR verification.

Lucky Draw

Randomly selects winners from eligible participants.

Reward Engine Rules

The engine must:

Prevent duplicate rewards.

Validate eligibility.

Check campaign status.

Verify stage completion.

Ensure reward budget remains available.

Create a reward record.

Initiate a NIM transfer via the organiser's funded reward pool workflow.

Update analytics.

Update Event Passport.

Notify the participant of success or failure.

---

16. QR & Claim Flow

QR codes are the primary interaction method for many campaign stages.

The system supports multiple QR strategies to suit different event types while reducing abuse.

QR Types

Static QR

Used for simple check-ins.

Dynamic QR

Regenerates every 15–30 seconds with a signed, time-limited token.

Ideal for conferences and workshops to discourage sharing outside the venue.

Hidden QR

Placed around a venue for treasure hunts.

Sponsor QR

Displayed only at sponsor booths.

Merchant QR

Printed on receipts or shown at checkout.

Personal QR

Unique to each participant for networking missions.

Claim Flow

User scans QR

↓

Mini App opens stage

↓

Wallet verified

↓

Campaign active?

↓

Stage active?

↓

Already claimed?

↓

Verification passes?

↓

Reward Engine validates

↓

Reward recorded

↓

Transaction initiated

↓

Passport updated

↓

Success screen

Claim Validation Rules

Before a claim is accepted, the backend must verify:

- Campaign exists and is live.
- Stage is currently active.
- Wallet is authenticated.
- Wallet has not already claimed the stage.
- Campaign budget is sufficient.
- QR token (if dynamic) is valid and unexpired.
- Stage-specific conditions (e.g. quiz score, secret code, referral, or merchant verification) are satisfied.

If any check fails, the user receives a clear explanation and, where possible, guidance on how to become eligible.

Error Handling

Common responses include:

- Campaign not started.
- Campaign has ended.
- Stage not yet unlocked.
- Reward pool exhausted.
- QR code expired.
- QR code already used.

- Duplicate claim detected.
- Wallet authentication required.
- Network temporarily unavailable.
- Transaction pending confirmation.

Each error state must be user-friendly, recoverable where appropriate, and logged for organisers to review in the analytics dashboard.

Part II Deliverables

By the end of Part II, the product specification defines:

- Complete functional scope of RallyNIM.
- Core modules and responsibilities.
- User stories for organisers, participants, sponsors, and merchants.
- End-to-end user journeys.
- Information architecture.
- Smart Campaign Template framework.
- Reward Engine behaviour.
- QR and claim validation workflows.

These requirements provide the foundation for implementation and will be referenced throughout the technical architecture in **Part III**, where the system design, MongoDB schema, REST API, Nimiq Mini App SDK integration (Testnet and Mainnet), and deployment architecture will be specified.

Part III – Technical Architecture

Version: 1.0

---

17. Technical Architecture Overview

Architecture Philosophy

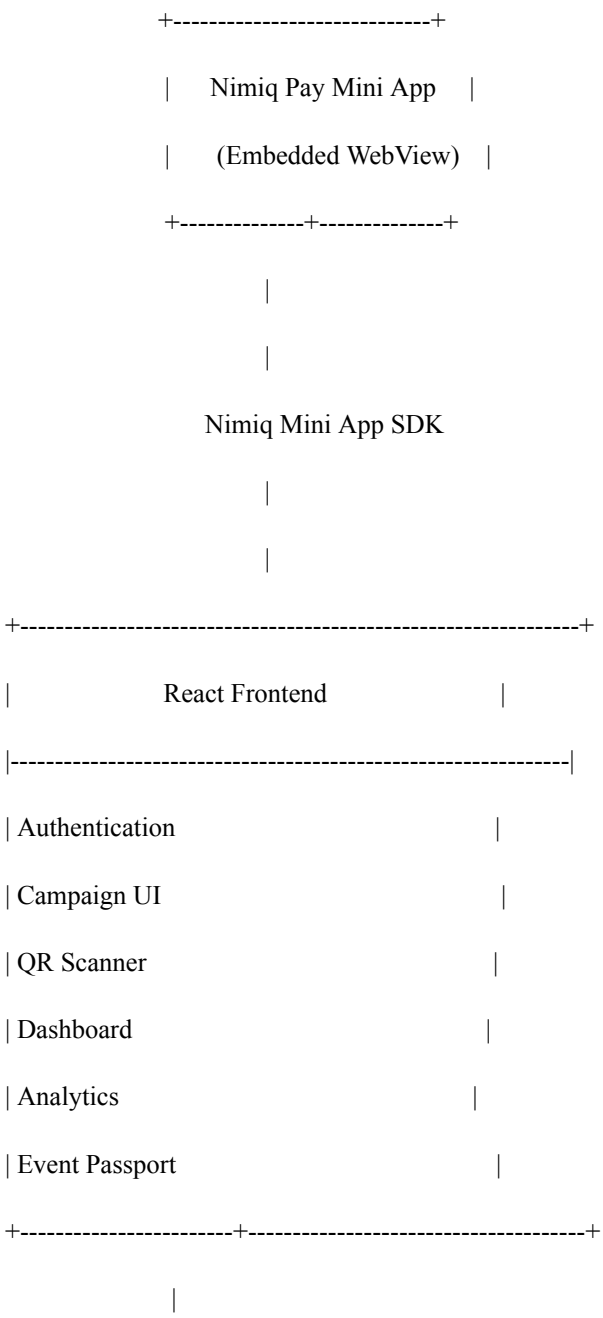
RallyNIM adopts a modern, scalable, API-first architecture that separates presentation, business logic, persistence, and blockchain interaction.

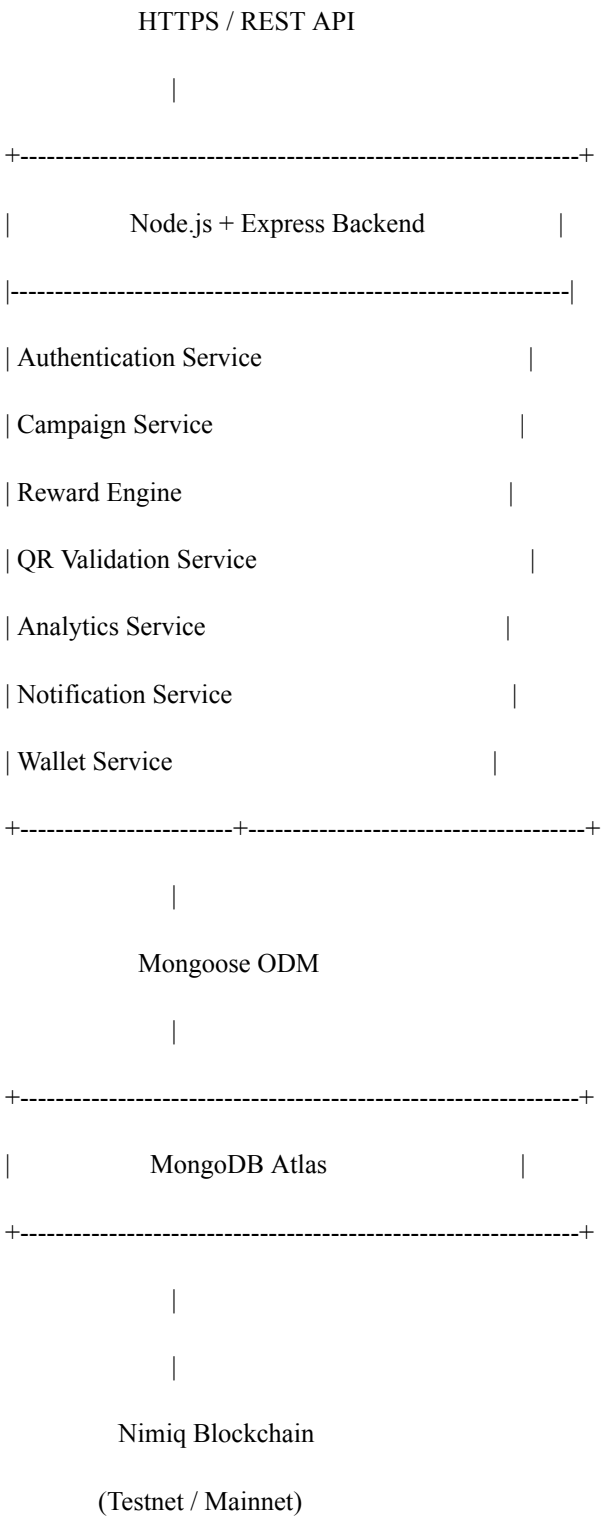
The architecture is designed to support:

- High availability

- Mobile-first performance
- Future multi-chain extensibility
- Secure wallet authentication
- Real-time event management
- Production deployment from day one

High-Level Architecture




HTTPS / REST API
Node.js + Express Backend
Authentication Service
Campaign Service
Reward Engine
QR Validation Service
Analytics Service
Notification Service
Wallet Service
Mongoose ODM
MongoDB Atlas
Nimiq Blockchain
(Testnet / Mainnet)

18. Technology Stack

Frontend

- React 19
- TypeScript
- Vite
- Tailwind CSS
- React Router
- React Query (TanStack Query)
- Zustand (Global State)
- Framer Motion
- React Hook Form
- Zod Validation
- Axios
- html5-qrcode
- qrcode

Backend

- Node.js
- Express.js
- TypeScript
- Mongoose
- JWT
- bcrypt (future admin authentication if needed)
- Helmet
- Cors
- Rate Limiter
- Morgan
- Winston Logger
- Zod
- Swagger

Database

MongoDB Atlas

Reason:

- Flexible schema

- Fast iteration
- Event-oriented data
- Excellent scalability
- Native JSON documents

DevOps

Frontend

Vercel

Backend

Railway / Render

Database

MongoDB Atlas

Storage

Cloudinary

Analytics

PostHog

Monitoring

Sentry

CI/CD

GitHub Actions

---

19. React Architecture

Design Principles

The frontend follows Feature-Based Architecture instead of component-based folders.

Each feature owns:

- components
- hooks
- pages
- services
- types

This scales much better.

Application Layers

Presentation

↓

Feature Modules

↓

Shared Components

↓

API Layer

↓

SDK Layer

↓

Backend

React Features

Authentication

Campaigns

Rewards

Claims

Passport

Templates

Dashboard

Analytics

Notifications

Settings

Global State

Use Zustand for:

- User
- Wallet
- Theme
- Notifications
- Campaign Cache
- Authentication

React Query manages:

- Server data
- Caching
- Optimistic updates
- Refetching

Routing

/

/login

/home

/campaigns

/campaign/:id

/rewards

/passport

/leaderboard

/dashboard

/dashboard/create

/dashboard/templates

/dashboard/analytics

/profile

/settings

---

20. Backend Architecture

Backend follows **Clean Architecture**.

Controllers

↓

Services

↓

Repositories

↓

Database

↓

Blockchain

Controllers

Receive HTTP Requests

Return Responses

No business logic.

Services

Contain business logic.

Examples

Campaign Service

Reward Service

Wallet Service

QR Service

Passport Service

Analytics Service

Repository Layer

Responsible for MongoDB operations.

No controller accesses MongoDB directly.

Utility Layer

Logging

Validation

Error Handling

JWT

Encryption

QR Generator

Date Utilities

---

21. MongoDB Database Design

Collections

users

campaigns

stages

claims

transactions

wallets

templates

passports

notifications

analytics

leaderboards

referrals

merchantRewards

sponsors

auditLogs

User Schema

{

_id,

walletAddress,

username,

avatar,

bio,

country,

role,

passportId,

createdAt,

updatedAt

}

Campaign Schema

{

_id,

title,

description,

banner,

category,

organizer,

rewardPool,

remainingPool,

status,

visibility,

startDate,

endDate,

location,

participants,

templateId,

createdAt,

updatedAt

}

Stage Schema

{

_id,

campaignId,

title,

description,

order,

rewardType,

rewardAmount,

verificationMethod,

status,

maximumClaims,

claimed,

startsAt,

endsAt

}

Claim Schema

{

_id,

campaignId,

stageId,

walletAddress,

reward,

status,

transactionHash,

claimedAt

}

Passport Schema

{

_id,

walletAddress,

eventsAttended,

campaignsCompleted,

totalNIMEarned,

badges,

achievements,

streak,

leaderboardRank

}

Template Schema

{

_id,

title,

category,

stages,

defaultRewardPool,

estimatedDuration,

recommendedAudience

}

Transaction Schema

{

_id,

walletAddress,

campaignId,

amount,

type,

status,

network,

transactionHash,

timestamp

}

Notification Schema

{

_id,

walletAddress,

title,

body,

type,

read,

createdAt

}

Analytics Schema

{

campaignId,

participants,

claims,

rewardDistributed,

completionRate,

topStage,

averageDuration

}

---

22. REST API Design

All endpoints begin with:

/api/v1

Authentication

POST /auth/connect

POST /auth/verify

POST /auth/refresh

POST /auth/logout

GET /auth/me

Campaigns

GET /campaigns

GET /campaign/:id

POST /campaign

PUT /campaign/:id

DELETE /campaign/:id

POST /campaign/:id/publish

POST /campaign/:id/pause

POST /campaign/:id/resume

POST /campaign/:id/archive

Stages

GET /campaign/:id/stages

POST /campaign/:id/stage

PUT /stage/:id

DELETE /stage/:id

Rewards

POST /reward/claim

POST /reward/verify

GET /reward/history

GET /reward/leaderboard

Templates

GET /templates

POST /templates

PUT /templates/:id

DELETE /templates/:id

Passport

GET /passport

GET /passport/:wallet

GET /passport/history

Analytics

GET /analytics/campaign/:id

GET /analytics/dashboard

GET /analytics/live

QR

POST /qr/generate

POST /qr/verify

POST /qr/rotate

---

23. Authentication Architecture

RallyNIM uses **wallet-native authentication**.

No passwords.

No email.

No usernames required.

Login Flow

User

↓

Connect Wallet

↓

Backend generates nonce

↓

User signs nonce

↓

Signature returned

↓

Backend verifies

↓

JWT issued

↓

Refresh Token stored

↓

Authenticated

JWT Payload

{

userId,

wallet,

role,

iat,

exp

}

Security

Access Token

15 minutes

Refresh Token

30 days

Rotate Refresh Token

Enabled

HTTP Only Cookies (where supported)

Enabled

CSRF Protection

Enabled for web deployments

Rate Limiting

Enabled

---

24. Nimiq Mini App SDK Integration

The Nimiq Mini App SDK is the primary interface between the Mini App and the user's wallet.

It is responsible for:

- Connecting the wallet
- Reading account information
- Requesting signatures
- Sending NIM transactions
- Checking balances
- Detecting network state

All wallet operations should be abstracted behind a dedicated `WalletService` so the rest of the application is isolated from SDK implementation details.

Wallet Service Interface

interface WalletService {

connect()

disconnect()

getAccount()

signMessage()

sendTransaction()

getBalance()

isConnected()

getNetwork()

switchNetwork()

}

Core SDK Operations

The application will use the SDK for:

- Connect wallet
- Retrieve active account
- Sign authentication nonce
- Send NIM funding transactions
- Send reward payout transactions (or confirm organiser-funded transactions, depending on the payout model)
- Retrieve balance before campaign creation
- Display transaction status and confirmations

Testnet Configuration

Environment:

NETWORK=testnet

Explorer links point to the Nimiq Testnet explorer.

Campaign funding uses Test NIM.

Reward distribution uses Test NIM.

Purpose:

- Feature development

- QA
- User Acceptance Testing
- Demo videos

No production funds are involved.

Mainnet Configuration

Environment:

NETWORK=mainnet

Explorer links switch to the Nimiq Mainnet explorer.

Campaign funding uses real NIM.

Reward distribution uses real NIM.

Before enabling Mainnet:

- Complete security review.
- Validate reward engine.
- Test all campaign templates on Testnet.
- Verify transaction handling and recovery.
- Ensure analytics and monitoring are operational.

---

25. Testnet → Mainnet Migration Plan

The application should be designed so that blockchain network selection is driven entirely by configuration rather than code changes.

Environment Variables

NETWORK=testnet

RPC_URL=...

EXPLORER_URL=...

API_BASE_URL=...

JWT_SECRET=...

MONGODB_URI=...

CLIENT_URL=...

Switching to production should require:

NETWORK=mainnet

and the corresponding production endpoints.

Migration Checklist

Phase 1 – Development

- Local React development
- Local Node.js server
- Local MongoDB or Atlas development cluster
- Nimiq Testnet
- Test wallets

Phase 2 – Internal Testing

- Deploy staging frontend
- Deploy staging backend
- Atlas staging database
- Testnet reward campaigns
- End-to-end QA
- Load testing

Phase 3 – Beta Launch

- Invite selected organisers
- Collect usability feedback
- Monitor transaction success rate
- Measure onboarding completion
- Fix identified issues

Phase 4 – Production

- Switch to Mainnet configuration
- Deploy production frontend
- Deploy production backend
- Production Atlas cluster
- Enable monitoring and alerts
- Begin live campaigns

No database schema changes should be required between Testnet and Mainnet. The only network-specific data stored in MongoDB should be the `network` field on transaction records.

---

26. Recommended Folder Structure

Frontend

src/

├── app/

├── assets/

├── components/

│ ├── common/

│ ├── layout/

│ ├── ui/

│

├── features/

│ ├── auth/

│ ├── campaigns/

│ ├── rewards/

│ ├── templates/

│ ├── passport/

│ ├── analytics/

│ ├── notifications/

│

├── hooks/

├── lib/

│ ├── api/

│ ├── sdk/

│ ├── utils/

│

├── pages/

├── routes/

├── services/

├── store/

├── styles/

├── types/

├── constants/

├── config/

└── main.tsx

Backend

server/

├── src/

│

├── config/

│

├── controllers/

│

├── services/

│

├── repositories/

│

├── middleware/

│

├── models/

│

├── routes/

│

├── validators/

│

├── interfaces/

│

├── types/

│

├── utils/

│

├── jobs/

│

├── events/

│

├── sockets/ (future real-time updates)

│

```
├── docs/            (Swagger)
│
├── tests/
│
├── app.ts
│
└── server.ts
```

---

27. Architecture Decisions

Decision	Rationale
React + Vite	Fast development, excellent performance, modern tooling
TypeScript	Strong typing, safer refactoring, improved maintainability
Tailwind CSS	Rapid UI development with a consistent design system
Node.js + Express	Lightweight, widely adopted, ideal for REST APIs
MongoDB Atlas	Flexible schema for campaigns, rewards, analytics and event data
Mongoose	Schema validation and clean data modelling

Zustand	Minimal, performant client-side state management
React Query	Server state caching, retries and optimistic updates
Wallet-native authentication	Removes passwords and aligns with Web3 best practices
Feature-based frontend structure	Scales more effectively as the application grows
Repository pattern	Separates persistence from business logic
Environment-driven network selection	Seamless transition from Testnet to Mainnet without code changes

---

Part III Deliverables

By the end of Part III, the engineering team has:

- A complete technical architecture.
- Frontend and backend design principles.
- MongoDB collection and schema definitions.
- REST API structure.
- Wallet-native authentication flow.
- Nimiq Mini App SDK integration strategy.
- Testnet-to-Mainnet migration plan.
- Recommended project folder structure.

This architecture provides a production-ready foundation while remaining flexible enough for future enhancements such as WebSockets for live dashboards, scheduled jobs for campaign automation, push notifications, and additional reward mechanisms.

The next section, **Part IV – UI/UX Specification**, will define the design system, screen-by-screen interface specifications, component library, responsive behaviour, animations, accessibility standards, and user experience guidelines required to deliver a polished, competition-ready Mini App.

Part IV – UI/UX Specification

Version: 1.0

> **Design Goal:** Build the most polished Mini App in the Nimiq ecosystem. Users should immediately feel that RallyNIM is a premium, production-ready application—not a hackathon prototype.

---

28. Design Philosophy

RallyNIM follows **Apple's Human Interface Guidelines**, **Material Design 3**, and **Linear's minimalist design principles**.

The UI must communicate three things immediately:

- Trust
- Speed
- Delight

Every interaction should feel effortless.

Core Design Principles

1. Mobile First

The Mini App runs inside Nimiq Pay.

Everything is designed for phones first.

Desktop simply adapts.

---

2. One-Handed Experience

Critical buttons remain within thumb reach.

Examples:

- Claim Reward
- Next Stage
- Join Campaign

- Scan QR

---

3. Zero Learning Curve

Every primary action should be obvious.

No documentation required.

---

4. Delight Through Motion

Animations should:

- reinforce actions
- provide feedback
- never distract

---

5. Beautiful by Default

Every screen should be worthy of a product showcase.

---

29. Design System

Brand Personality

RallyNIM should feel:

- Modern
- Energetic
- Friendly
- Trustworthy
- Rewarding

Colour Palette

Primary

Emerald Green

#10B981

Represents:

Rewards

Growth

Success

NIM ecosystem

Secondary

Royal Blue

#2563EB

Represents:

Trust

Technology

Security

Accent

Gold

#F59E0B

Represents:

Achievements

Badges

Premium rewards

Success

#22C55E

Error

#EF4444

Warning

#F97316

Info

#3B82F6

Background

Light

#F8FAFC

Dark

#0F172A

Surface

#FFFFFF

Dark

#1E293B

Typography

Font

Inter

Fallback

System Fonts

Font Scale

Element	Size	Weight
Hero	40px	Bold
H1	32px	Bold
H2	28px	SemiBold
H3	24px	SemiBold
H4	20px	Medium
Body	16px	Regular
Caption	14px	Regular
Label	12px	Medium

Border Radius

Buttons

16px

Cards

24px

Inputs

14px

Badges

999px

Shadows

Soft shadows only.

Avoid heavy elevation.

Spacing System

Based on 8px.

4

8

16

24

32

48

64

96

---

30. Icons

Library

Lucide React

Reasons

Minimal

Modern

Tree-shakable

Consistent

Icon Style

Outline

2px stroke

Rounded

Never filled

---

31. Mobile UX Guidelines

Navigation

Bottom Navigation

🏠 Home

🎁 Rewards

📷 Scan

🏅 Passport

👤 Profile

Maximum:

5 tabs.

Floating Action Button

Only organisers see:

+

Create Campaign

Thumb Zones

Primary buttons remain in bottom third.

Pull to Refresh

Supported on:

Campaigns

Dashboard

Passport

Leaderboard

Analytics

Infinite Scroll

Campaigns

Rewards

Notifications

History

---

32. Screen Specifications

Splash Screen

Purpose

Brand loading

Elements

Logo

Animated gradient

Loading indicator

Duration

<2 seconds

Onboarding Screen

Purpose

Explain value.

Sections

1.

Reward participation.

2.

Earn NIM.

3.

Join events.

CTA

Connect Wallet

Wallet Connection

Elements

Wallet Status

Benefits

Connect Button

Security Notice

Home Screen

Good Morning, Precious 👋

Upcoming Campaigns

Continue Campaign

Featured Events

Quick Actions

Recent Rewards

Leaderboard Preview

Quick Actions

Create Campaign

Scan QR

My Passport

Templates

Campaign Feed

Cards contain

Banner

Reward Pool

Participants

Time Left

Difficulty

Category

Join Button

Campaign Details

Hero Banner

Description

Reward Pool

Progress

Current Stage

Remaining Rewards

Timeline

Participants

CTA

Join Campaign

Active Stage

Large Stage Card

Workshop Challenge

5 NIM

Active

Ends in 14 min

Claim

Countdown

Animated progress

QR Scanner

Fullscreen

Large Camera

Flash

Gallery Upload

Scanning Animation

Success Animation

Reward Success Screen

Large celebration.

🎉

Congratulations!

You earned

5 NIM

Buttons

Share

Continue

Passport

Reward History

Grouped by date.

Each card

Reward

Campaign

Amount

Status

Transaction

Passport

Hero

Level

Total NIM

Achievements

Badges

Events

Timeline

Streak

Leaderboard

Leaderboard

Top 3

Large podium.

Everyone else

Rank list.

Organiser Dashboard

Overview Cards

Active Campaigns

Revenue

Rewards

Participants

Analytics

Campaign Builder

Multi-step wizard.

Step 1

Choose Template

↓

Step 2

Details

↓

Step 3

Stages

↓

Step 4

Rewards

↓

Step 5

Funding

↓

Step 6

Preview

↓

Publish

Analytics Dashboard

Charts

Daily Claims

Wallet Growth

Rewards

Completion

Heatmap

Peak Hours

Notifications

Grouped.

Unread

Read

Campaign

Rewards

Updates

Profile

Avatar

Wallet

Username

Statistics

Achievements

Settings

Logout

Settings

Theme

Language

Notifications

Security

Help

About

Privacy

---

33. Component Library

Reusable components.

Buttons

Primary

Secondary

Ghost

Danger

Loading

Icon

Floating

Cards

Campaign Card

Reward Card

Passport Card

Analytics Card

Stage Card

Template Card

Notification Card

Inputs

Text

Textarea

Select

Date

Time

Search

Wallet

Amount

Navigation

Bottom Nav

Tabs

Breadcrumbs

Stepper

Pagination

Feedback

Toast

Alert

Modal

Drawer

Tooltip

Popover

Snackbar

Loading

Spinner

Skeleton

Progress Bar

Shimmer

Data Display

Table

Timeline

Charts

Leaderboard

Statistics

Badge

Chip

Avatar

QR Components

QR Generator

QR Scanner

QR Preview

Dynamic QR

QR Countdown

---

34. Empty States

Every empty state should encourage action rather than simply stating that no data exists.

No Campaigns

Illustration

🚀

Title

"No Campaigns Yet"

Description

"Create your first campaign and start rewarding your community."

Button

Create Campaign

No Rewards

🎁

"You haven't earned any rewards yet."

Button

Browse Campaigns

No Notifications

🔔

"You're all caught up."

No Passport History

🏅

"Attend your first event to begin building your Event Passport."

Search Empty

🔍

"No campaigns matched your search."

No Internet

📶

"No connection."

Retry Button

---

35. Error States

Error handling should be friendly, actionable, and never expose technical details to users.

Wallet Connection Failed

Title

Couldn't Connect

Message

Please try reconnecting your wallet.

Button

Retry

Authentication Failed

Message

Your session has expired.

Button

Reconnect Wallet

QR Expired

Title

QR Code Expired

Button

Scan Again

Already Claimed

✅

"You've already claimed this reward."

Campaign Ended

⏰

"This campaign has ended."

Stage Locked

🔒

"Complete previous stages to unlock this one."

Reward Pool Empty

💰

"All available rewards have been claimed."

Transaction Failed

Title

Transaction Failed

Buttons

Retry

Contact Support

Server Error

Friendly Illustration

"We're having a temporary issue. Please try again in a moment."

404

Illustration

Compass

Message

"Looks like you're lost."

---

36. Animations

Animations should improve comprehension, not distract.

Target duration:

150–300ms for standard interactions.

Page Transitions

Fade + Slide

Campaign Cards

Lift on hover (desktop)

Scale on tap (mobile)

Buttons

Press animation

95% scale

Bounce back

Reward Claim

Confetti burst

Coin animation

Success sound (optional)

Reward counter increments

Passport badge glow

QR Scanner

Animated scan line

Pulse effect while scanning

Green flash on successful scan

Subtle vibration on supported devices

Progress Indicators

Animated circular countdowns for active stages.

Linear progress bars for campaign completion.

Leaderboard

Top three animate onto the podium with staggered entrance effects.

Rank changes smoothly transition rather than jumping.

Event Passport

New badges unlock with a flip animation.

Achievement cards slide into view.

Streak counter animates upward.

Loading States

Replace blank screens with skeleton placeholders.

Use shimmer effects for lists and cards.

Avoid spinners when content structure is known.

Micro-interactions

- Heartbeat pulse for live campaigns.
- Wallet icon animates after successful connection.
- NIM balance counts up when updated.
- Notification badge gently pulses when unread items arrive.
- Campaign cards subtly glow when a new stage becomes available.

---

37. Accessibility

Objective

RallyNIM is designed to be accessible to everyone, regardless of ability, device, or environment.

Accessibility is considered a core product requirement rather than an optional enhancement.

The application should meet **WCAG 2.2 AA** standards wherever technically feasible.

Accessibility Principles

The product follows four principles:

Perceivable

Information must be presented in ways everyone can perceive.

Examples:

- High colour contrast
- Icons paired with text
- Images include descriptions
- QR instructions include text

Operable

Every feature should be usable without requiring complex gestures.

Examples:

- Large touch targets
- Clear navigation
- Keyboard support
- Logical focus order

Understandable

Users should immediately understand:

- what happened
- what to do next
- why something failed

No technical jargon.

Robust

The Mini App should work correctly across:

- Android
- iOS
- Different screen sizes
- Screen readers
- Future browser updates

Colour Accessibility

Text contrast should satisfy WCAG AA.

Minimum ratios:

Normal text

4.5 : 1

Large text

3 : 1

Icons

3 : 1

Never communicate information using colour alone.

Instead of:

🟢 Active

Use:

🟢 Active

Instead of:

🔴 Error

Use:

❌ Error

Typography Accessibility

Minimum font size

16px

Body text

Preferred line height

1.5

Maximum paragraph width

75 characters

Avoid:

- thin fonts
- condensed fonts
- decorative fonts

Touch Accessibility

Minimum touch target

44 × 44 px

Preferred

48 × 48 px

Spacing between buttons

8px minimum

Screen Reader Support

Every interactive element requires accessible labels.

Example

Instead of

<button>

Use

<button aria-label="Claim Reward">

Required Labels

Connect Wallet

Create Campaign

Join Campaign

Scan QR

Claim Reward

Open Passport

View Analytics

Delete Campaign

Pause Campaign

Keyboard Navigation

Although the Mini App is mobile-first, desktop users should be fully supported.

Required:

Tab navigation

Visible focus indicators

Enter activates buttons

Escape closes dialogs

Arrow key navigation where appropriate

Focus Management

Every modal should:

Move focus inside the modal when opened.

Return focus to the triggering element when closed.

Prevent keyboard focus from escaping the modal while it is open.

Motion Accessibility

Some users prefer reduced motion.

Respect the operating system preference:

prefers-reduced-motion

When enabled:

- Disable confetti
- Remove large transitions
- Reduce parallax
- Shorten animations
- Keep essential feedback only

Visual Accessibility

Avoid flashing content.

Never exceed:

3 flashes per second

Support:

Dark Mode

Light Mode

High Contrast Mode (future enhancement)

QR Accessibility

Scanning instructions should never rely only on visuals.

Example:

Instead of

"Scan the QR."

Use

"Point your camera at the event QR code to claim your reward."

Forms Accessibility

Every field must include:

Visible label

Placeholder (optional, not a replacement)

Error message

Helper text when needed

Example:

Wallet Name

[_____]

Choose a name that helps identify this wallet.

Error Accessibility

Every error message should:

Explain what happened.

Explain why.

Explain how to fix it.

Example

❌ Poor

Error 401

✅ Better

Your wallet session has expired.

Reconnect your wallet to continue.

Loading Accessibility

During loading:

Use skeleton placeholders where possible.

Provide loading announcements for assistive technologies.

Example

Loading campaigns...

instead of only showing a spinner.

Images & Illustrations

Every meaningful image should include descriptive alternative text.

Examples:

- Event banner: "Banner for Web3 Conference 2026"
- Reward badge: "Gold Community Champion badge"

Decorative graphics should be ignored by screen readers.

Charts & Analytics

Charts must not rely solely on colour.

Provide:

- labels
- legends
- values
- table alternative for screen readers

Accessibility Testing

Accessibility testing should be included in the QA process.

Recommended tools:

- Lighthouse Accessibility Audit
- axe DevTools
- WAVE
- Manual keyboard navigation
- Android TalkBack
- iOS VoiceOver

Accessibility Acceptance Criteria

The product should satisfy the following minimum requirements before production release:

Requirement	Target
WCAG Compliance	AA
Lighthouse Accessibility	≥95
Keyboard Navigation	100%
Screen Reader Labels	100% Interactive Elements
Touch Target Size	≥44×44 px
Colour Contrast	WCAG AA
Focus Indicators	All Interactive Components
Reduced Motion Support	Enabled
Form Labels	100%
Accessible Error Messages	100%

Accessibility Checklist

Before every release, verify that:

- ✓ All buttons have accessible labels.
- ✓ Forms include visible labels and descriptive error messages.
- ✓ Text meets WCAG AA colour contrast requirements.
- ✓ Interactive elements meet minimum touch target sizes.
- ✓ Users can navigate with a keyboard where applicable.
- ✓ Screen readers correctly announce key actions and page changes.
- ✓ Motion respects the user's reduced-motion preference.
- ✓ Charts and visualisations provide non-colour alternatives.
- ✓ Images have appropriate alternative text or are marked decorative.
- ✓ No critical user flow depends solely on colour, sound, or animation.

This accessibility standard ensures that RallyNIM is inclusive, production-ready, and aligned with modern software quality expectations while strengthening its overall UX quality for the competition.

---

38. Responsive Behaviour

Primary target widths:

- **320–480px:** Small phones
- **481–768px:** Large phones
- **769–1024px:** Tablets
- **1025px+:** Desktop administration

Desktop is intended primarily for organisers managing campaigns and analytics, while participants are expected to use the Mini App on mobile.

---

39. UX Success Criteria

The UI should satisfy the following measurable goals:

Metric	Target

Time to first reward claim	< 60 seconds
Wallet connection success	> 98%
Campaign creation time (using template)	< 2 minutes
QR scan to reward confirmation	< 10 seconds
First-time task completion	> 90%
Lighthouse Performance Score	≥ 95
Lighthouse Accessibility Score	≥ 95
Lighthouse Best Practices Score	≥ 95
Lighthouse SEO Score (web landing page)	≥ 90

Part IV Deliverables

By the end of Part IV, the product team has a complete UI/UX specification covering:

- A consistent visual design system.
- Mobile-first interaction principles.
- Screen-by-screen layouts for participant and organiser journeys.
- A reusable component library.
- Comprehensive empty and error state guidance.

- Animation and micro-interaction specifications.
- Accessibility and responsive design standards.

This specification is intended to ensure that RallyNIM delivers a polished, app-store-quality experience that directly supports the competition's scoring criteria for **Design & UX**, while also reinforcing ease of use, repeat engagement, and trust.

Part V – Security & Production

Version: 1.0

> **Objective:** Build RallyNIM as a production-grade application that organisers trust with real funds. Security is a product feature, not an afterthought. Every NIM transaction, campaign, and reward claim must be protected against abuse while maintaining a fast and seamless user experience.

---

40. Security Architecture

Security Philosophy

RallyNIM follows a **Zero Trust** architecture.

Every request must be verified.

Never trust:

- Wallet addresses
- QR codes
- Client-side validation
- Browser state
- Request parameters

All sensitive validation occurs on the backend.

Security Layers

User

↓

Wallet Authentication

↓

JWT Validation

↓

API Authorization

↓

Business Rule Validation

↓

Fraud Detection

↓

Rate Limiting

↓

Database

↓

Blockchain

Security Principles

Authenticate Everything

Every API request requires authentication unless explicitly public.

Validate Everything

Never trust frontend data.

Every field must be validated.

Principle of Least Privilege

Users only receive permissions required for their role.

Roles:

- Participant
- Organiser
- Admin
- Super Admin (future)

Defence in Depth

Multiple independent security mechanisms protect every transaction.

---

41. Fraud Prevention

Reward abuse is the biggest threat to RallyNIM.

The platform must assume users will attempt to exploit reward campaigns.

Fraud Detection Engine

Every claim passes through a Fraud Detection Service before the Reward Engine executes.

Claim Request

↓

Authentication

↓

Wallet Verification

↓

Campaign Validation

↓

Stage Validation

↓

QR Validation

↓

Duplicate Check

↓

Velocity Check

↓

Fraud Score

↓

Approved / Rejected

Fraud Rules

Duplicate Claims

Prevent users from claiming the same reward twice.

Validation:

- Wallet address
- Campaign
- Stage
- Transaction history

QR Reuse

Each QR token may only be redeemed according to campaign rules.

Dynamic QR codes expire automatically.

Device Velocity

Flag users making excessive claim attempts.

Example:

- More than 20 failed claims in 5 minutes.
- More than 10 QR scans within 30 seconds.

Wallet Velocity

Detect abnormal behaviour.

Examples:

- Dozens of claims within seconds.
- Multiple campaigns claimed simultaneously beyond realistic limits.

Campaign Budget Protection

Before every reward:

- Verify remaining budget.
- Lock funds atomically.
- Prevent race conditions during concurrent claims.

Location Rules (Future)

Optional organiser setting.

Users must be within a defined radius of the venue to claim location-restricted rewards.

Time Window Validation

Claims only succeed if:

- Campaign is live.
- Stage is active.
- Reward has not expired.

Referral Abuse Detection

Reject:

- Self-referrals.
- Circular referrals.
- Duplicate wallet referrals.

Manual Review Queue

Suspicious activity enters a review queue rather than being paid immediately.

Examples:

- Extremely high claim velocity.
- Repeated QR failures.
- Multiple wallets linked to identical behaviour.

Fraud Score

Every claim receives a fraud score.

Score	Action
0–20	Automatically approve
21–50	Approve and log for monitoring
51–80	Hold for manual review
81–100	Reject and flag account

---

42. Dynamic QR Security

Dynamic QR codes are the primary defence against screenshot sharing and unauthorised reward claims.

QR Types

Static QR

- Single generated code.
- Best for low-risk events.
- Easy to print.
- Not recommended for high-value rewards.

Dynamic QR

Changes every **15–30 seconds**.

Recommended for:

- Conferences
- Workshops
- Sponsor booths
- Hackathons

Dynamic QR Payload

Each QR contains:

```
{

 "campaignId": "...",

 "stageId": "...",

 "nonce": "...",

 "expiresAt": "...",
```

```
  "signature": "...",
  "version": 1
}
```

The payload is cryptographically signed by the backend.

Clients must never generate trusted QR payloads.

QR Lifecycle

Create QR

↓

Backend Signs Payload

↓

Display QR

↓

User Scans

↓

Backend Verifies Signature

↓

Validate Expiry

↓

Validate Campaign

↓

Reward Engine

QR Expiration

Default lifetime:

20 seconds

Organisers may configure:

- 15 seconds
- 20 seconds
- 30 seconds
- 60 seconds (low-security events)

Anti-Screenshot Protection

Dynamic QR reduces:

- Social sharing
- Screenshot reuse
- Remote claiming
- Replay attacks

---

43. Rate Limiting

Every public endpoint should implement rate limiting.

Authentication

POST /auth/verify

10 requests

per minute

per IP

QR Verification

POST /qr/verify

30 requests

per minute

per wallet

Reward Claims

POST /reward/claim

10 claims

per minute

per wallet

Campaign Creation

POST /campaign

5 requests

per minute

per organiser

Analytics

GET /analytics

60 requests

per minute

API Abuse

If limits are exceeded:

Return:

429 Too Many Requests

Include:

- Retry-After header
- Human-readable error message

---

44. Wallet Verification

Wallet verification is the foundation of identity.

Authentication Flow

Connect Wallet

↓

Backend Generates Nonce

↓

Wallet Signs Nonce

↓

Backend Verifies Signature

↓

JWT Generated

↓

Authenticated

Wallet Ownership

A wallet is considered verified only if:

- Signature matches the supplied address.
- Nonce has not expired.
- Nonce has not been reused.

Session Management

Access Token

15 minutes

Refresh Token

30 days

Automatic refresh before expiry.

Wallet Security Rules

- Nonce expires after 5 minutes.
- Nonce is single-use.
- Refresh tokens rotate after each use.
- Sessions can be revoked server-side.

Organiser Verification (Future)

Future versions may require enhanced verification for organisers running large campaigns.

Possible checks:

- Verified organiser badge.
- Organisation profile.
- Optional KYC.
- Funding history.

---

45. Data Protection

Sensitive information stored:

- Wallet address
- Profile information
- Analytics
- Campaign data

Passwords are not stored because authentication is wallet-based.

Encryption

At Rest

MongoDB Atlas encryption enabled.

In Transit

TLS 1.3.

Secrets

Stored in environment variables or a managed secrets service.

JWT

Signed using a strong secret.

Automatic expiry.

Rotation supported.

Input Validation

Every request validated using Zod.

Reject:

- Invalid wallet addresses.
- Negative reward values.
- Invalid campaign dates.
- Malformed JSON.

- Unexpected fields.

---

46. Performance

The application should remain responsive during live events with hundreds or thousands of concurrent participants.

Performance Targets

Metric	Target
First Contentful Paint	< 1.5 s
Largest Contentful Paint	< 2.5 s
Time to Interactive	< 3 s
API Response Time (P95)	< 250 ms
QR Verification	< 500 ms
Reward Claim End-to-End	< 5 s
Wallet Authentication	< 3 s

Frontend Optimisation

- Code splitting
- Lazy-loaded routes
- Image optimisation
- Tree shaking
- Route prefetching
- React Query caching
- Skeleton loading

Backend Optimisation

- MongoDB indexes
- Efficient aggregation pipelines
- Connection pooling
- Compression
- Pagination
- Background jobs for non-critical tasks

Database Indexes

Recommended indexes:

users.walletAddress

campaigns.status

campaigns.organizer

claims.walletAddress

claims.stageId

transactions.walletAddress

analytics.campaignId

Caching

Cache:

- Campaign templates
- Live campaign metadata

- Public campaign listings

Avoid caching:

- Wallet balances
- Reward eligibility
- Authentication state

---

47. Logging

Every critical action must be logged.

Logs support:

- Debugging
- Security investigations
- Analytics
- Compliance

Log Levels

INFO

WARN

ERROR

DEBUG (development only)

Events to Log

Authentication

Campaign creation

Campaign updates

Reward claims

QR validation

Transaction submission

Transaction confirmation

Fraud detection

API errors

Unexpected exceptions

Log Format

Structured JSON.

Example:

{

"timestamp": "...",

"level": "INFO",

"service": "RewardService",

"wallet": "...",

"campaignId": "...",

"event": "RewardClaimed"

}

Audit Log

Immutable audit records for organiser actions:

- Campaign created
- Campaign edited
- Campaign published
- Reward pool funded
- Campaign archived

Audit logs should never be editable.

---

48. Monitoring

Production monitoring should provide real-time visibility into application health.

Error Monitoring

Tool:

Sentry

Track:

- Frontend crashes
- Backend exceptions
- Unhandled promise rejections
- API failures

Analytics Monitoring

Track:

- Daily Active Wallets
- Campaign creation rate
- Claim success rate
- Average claim time
- Wallet connection success
- Reward distribution totals

Infrastructure Monitoring

Monitor:

- CPU
- Memory

- Disk
- Network
- Database connections
- API latency

Alerts

Notify operators when:

- Error rate exceeds threshold.
- API latency exceeds target.
- Reward claim failures spike.
- Database connectivity is lost.
- Blockchain transaction failures increase.
- Campaign funding transactions fail.

Health Checks

Expose:

GET /health

Returns:

- API status
- Database connectivity
- Blockchain connectivity
- SDK availability
- Application version

---

49. Deployment

The deployment process should support rapid iteration while protecting production stability.

Environments

Phase 1 — Local Development

- React (Vite) running locally
- Node.js/Express running locally
- MongoDB Atlas (or local MongoDB)
- Nimiq **Testnet**
- Test wallets

This is where you'll build and test all features.

Phase 2 — Production Demo (Competition Submission)

Deploy directly to production services:

- **Frontend:** Vercel
- **Backend:** Railway or Render
- **Database:** MongoDB Atlas
- **Blockchain:** Nimiq Testnet (until you're ready for Mainnet)

This is sufficient for the judges to test your Mini App.

Phase 3 — Mainnet Launch

When you're confident everything works:

- Change your environment variables:

  NETWORK=mainnet

- Update the RPC/explorer configuration.
- Use a real organiser wallet with real NIM.
- Deploy the same codebase—no code changes should be needed if you've followed the environment-based configuration.

Deployment Pipeline

Developer Push

↓

GitHub

↓

GitHub Actions

↓

Lint

↓

Tests

↓

Build

↓

Deploy to Vercel (Frontend)

↓

Deploy to Railway (Backend)

↓

MongoDB Atlas

↓

Nimiq Testnet (or Mainnet)

Environment Variables

Backend

NODE_ENV=production

PORT=5000

MONGODB_URI=

JWT_SECRET=

REFRESH_SECRET=

CLIENT_URL=

NETWORK=mainnet

RPC_URL=

EXPLORER_URL=

SENTRY_DSN=

Frontend

VITE_API_URL=

VITE_NETWORK=

VITE_APP_NAME=RallyNIM

VITE_SENTRY_DSN=

Backup Strategy

Database:

- Daily automated backups.
- Point-in-time recovery (Atlas).
- Backup retention based on production requirements.

Disaster Recovery

Recovery objectives:

- **RPO (Recovery Point Objective):** ≤ 15 minutes.
- **RTO (Recovery Time Objective):** ≤ 1 hour.

Production Checklist

Before enabling Mainnet:

- ✓ All unit tests passing.
- ✓ Integration tests passing.

- ✓ Manual QA completed.
- ✓ Security review completed.
- ✓ Wallet authentication verified.
- ✓ Reward engine validated.
- ✓ Dynamic QR tested under load.
- ✓ Monitoring enabled.
- ✓ Logging enabled.
- ✓ Backups configured.
- ✓ Environment variables verified.
- ✓ Testnet sign-off completed.

---

50. Future Production Enhancements

Planned enterprise features:

- Web Application Firewall (WAF).
- Multi-region deployments.
- Redis caching layer.
- WebSocket support for live campaign updates.
- Queue workers (BullMQ) for asynchronous reward processing.
- Push notifications.
- Multi-signature organiser wallets.
- Geographic claim validation.
- Advanced fraud detection using behavioural analytics.
- Role-based administration dashboard.
- Automatic campaign scheduling.
- API versioning and rate-limit tiers.

Part V Deliverables

By the end of Part V, the engineering team has a complete production and security blueprint covering:

- Defence-in-depth security architecture.
- Fraud prevention and claim validation.
- Dynamic QR security model.
- Wallet verification and session management.
- API rate limiting.
- Performance targets and optimisation strategy.
- Structured logging and immutable audit trails.
- Monitoring, alerting, and health checks.
- Deployment pipeline, backup strategy, and disaster recovery planning.

This specification ensures that RallyNIM is capable of securely handling real NIM rewards, organiser-funded campaigns, and high-volume live events while maintaining the reliability, performance, and trust expected of a production-grade Mini App.

Part VI – Competition Winning Strategy

Version: 1.0

> **Objective:** RallyNIM is designed not only to be an excellent product but to maximise its score across all **105 judging points**. Every feature, launch activity, and marketing effort should directly contribute to the competition rubric.

---

51. Winning Strategy Overview

Competition Goal

Win **1st Place ($10,000)** by optimising across:

- Design & UX (25)
- Functionality (25)
- Usefulness & Originality (25)
- Marketing & Distribution (25)
- Bonus (5)

Rather than building unnecessary features, RallyNIM prioritises delivering a polished, complete experience with measurable user engagement.

---

52. Feature-to-Judging Rubric Mapping

Design & UX (25 Points)

Judging Criteria	RallyNIM Feature

First Impression	Premium landing experience, polished UI, branded onboarding
Visual Design	Modern design system, gradients, animations, consistent spacing
Navigation	Bottom navigation, intuitive flows, campaign wizard
Mobile Experience	Mobile-first responsive design optimised for MiniPay
Onboarding	Wallet connect and first campaign join in under 60 seconds

Target Score: 25/25

Functionality (25 Points)

Judging Criteria	RallyNIM Feature
Core Feature	Campaign creation and reward claiming
Nimiq Integration	Wallet authentication, signatures, NIM transactions
Speed	Lazy loading, React Query caching, optimised APIs
Error Handling	Friendly error states and recovery flows
Completeness	End-to-end organiser and participant experience

Target Score: 25/25

Usefulness & Originality (25 Points)

Judging Criteria	RallyNIM Feature
Problem Solved	Simplifies reward distribution for communities and events
Target Audience	Event organisers, hackathons, merchants, universities, DAOs
Originality	Smart campaign templates, Event Passport, dynamic rewards
Repeat Value	Users return for new campaigns, streaks, badges, leaderboards
Ecosystem Value	Makes MiniPay useful beyond simple payments

Target Score: 25/25

Marketing & Distribution (25 Points)

Judging Criteria	Planned Activities
Unique Users	Onboard communities to test campaigns
User Acquisition	X, LinkedIn, Telegram, Discord, WhatsApp communities
Content	Build logs, videos, tutorials, launch thread

Community Engagement	Join builder calls, answer questions, share progress
Submission Quality	Professional demo, polished screenshots, documentation

Target Score: 25/25

Bonus (5 Points)

Incentivising NIM Usage

RallyNIM naturally encourages NIM adoption by:

- Funding campaign reward pools with NIM.
- Distributing all campaign rewards in NIM.
- Rewarding referrals in NIM.
- Running NIM-based treasure hunts.
- Supporting merchant cashback campaigns in NIM.
- Encouraging repeat participation through NIM rewards.

Target Score: 5/5

---

53. Marketing Plan

Marketing starts before launch.

The goal is to create awareness while gathering real users and feedback.

Phase 1 – Pre-Build (Week 1)

Objectives:

- Announce the project.
- Explain the problem.
- Join the builder community.
- Share the roadmap.

Content:

- "Why I'm building RallyNIM"
- Wireframes
- Brand reveal
- Initial architecture

Phase 2 – Build in Public (Weeks 2–3)

Post consistently across social platforms.

Content ideas:

- Daily development updates
- UI previews
- GIFs of new features
- Before vs after improvements
- Technical insights
- User feedback

Recommended frequency:

- X: 1–2 posts/day
- LinkedIn: 2–3 posts/week
- Builder community: Daily updates

Phase 3 – Launch (Week 4)

Launch assets:

- Product teaser
- Launch trailer (30–60 seconds)
- Demo video
- Screenshots
- Landing page
- Feature infographic
- FAQ
- Submission package

---

54. Build in Public Strategy

The judges value active builders.

Build publicly from day one.

Weekly Content Plan

Monday

Development goals.

Tuesday

Feature demo.

Wednesday

Technical insight.

Thursday

Progress recap.

Friday

Community feedback.

Saturday

Behind-the-scenes.

Sunday

Weekly summary.

Content Types

- UI screenshots
- Short demo videos
- Code snippets
- Architecture diagrams
- Feature announcements
- Polls
- User testimonials
- Milestone celebrations

Community Engagement

Participate by:

- Attending builder calls.
- Answering community questions.
- Helping other participants.
- Sharing lessons learned.
- Requesting feedback early and often.

---

55. Analytics Strategy

Success should be measurable.

Product Metrics

Track:

- Wallet connections
- New users
- Returning users
- Campaigns created
- Campaigns joined

- Rewards claimed
- NIM distributed
- QR scans
- Passport completions
- Referral conversions

Funnel

Visitor

↓

Wallet Connected

↓

Joined Campaign

↓

Claimed Reward

↓

Completed Campaign

↓

Returned

Success Targets

Metric	Target
Wallet Connection Rate	> 90%
Campaign Join Rate	> 70%
Reward Claim Success	> 95%
Campaign Completion	> 60%
Returning Users	> 40%
Average Session Duration	> 5 minutes

Event Tracking

Track events such as:

- Wallet connected
- Campaign viewed
- Campaign joined
- QR scanned
- Reward claimed
- Passport updated
- Campaign created

- Campaign published
- Referral completed

---

56. Demo Strategy

The demo should tell a story, not just show features.

Total duration:

3–5 minutes

Demo Flow

Scene 1 – The Problem (20 seconds)

Introduce the difficulty of distributing rewards fairly at events.

Scene 2 – RallyNIM Solution (30 seconds)

Show the homepage and explain how organisers create campaigns.

Scene 3 – Organiser Journey (60 seconds)

- Create a campaign.
- Select a smart template.
- Configure stages.
- Fund reward pool.
- Publish.

Scene 4 – Participant Journey (60 seconds)

- Connect wallet.
- Join campaign.
- Scan QR.
- Claim reward.

- View Event Passport update.

Scene 5 – Analytics (30 seconds)

Show:

- Live dashboard
- Claims
- Participants
- Reward distribution

Scene 6 – Ecosystem Impact (30 seconds)

Explain how RallyNIM increases NIM usage and makes MiniPay more valuable for organisers and communities.

Demo Checklist

- Clean database.
- Stable internet connection.
- Test wallet with sufficient Test NIM.
- Pre-created campaign ready.
- Backup recording available.
- Browser cache cleared.
- Notifications enabled.

---

57. Launch Checklist

Product

- All critical bugs fixed.
- Responsive layouts verified.
- Wallet connection tested.
- Reward engine validated.
- Dynamic QR tested.
- Analytics working.
- Accessibility checks completed.

Documentation

- README completed.
- Architecture documented.
- API documentation available.
- Setup guide written.
- Environment variables documented.

Marketing Assets

- Logo
- Screenshots
- Hero image
- Demo video
- Product description
- Feature graphics
- Social banners

Competition Submission

- GitHub repository public.
- Live Mini App URL.
- Demo video.
- Project description.
- Installation instructions.
- Test credentials (if required).
- Clear explanation of NIM integration.

Final Quality Assurance

- No console errors.
- No broken links.
- No placeholder content.
- Mobile experience verified.
- Loading states implemented.
- Error states tested.
- Lighthouse scores meet targets.

---

58. Product Roadmap

Phase 1 – Competition MVP

- Wallet authentication
- Campaign management
- Smart templates
- Dynamic QR claims
- Reward engine
- Event Passport
- Analytics dashboard
- Mobile-first UI

Phase 2 – Public Beta

- Push notifications
- Referral campaigns
- Merchant cashback
- Leaderboards
- Advanced organiser analytics
- Custom campaign templates

Phase 3 – Production Growth

- Team workspaces
- Multi-organiser accounts
- Scheduled campaigns
- Advanced fraud detection
- Geographic claim verification
- WebSocket-powered live updates
- API for third-party integrations

Phase 4 – Ecosystem Expansion

- NFT achievement badges

- Cross-community campaign discovery
- Sponsor marketplace
- Multi-language support
- AI-assisted campaign builder
- Community template marketplace
- Enterprise reporting and exports

---

59. Definition of Success

RallyNIM will be considered successful if it:

Competition Success

- Achieves top-tier scores across all five judging categories.
- Demonstrates polished UX and reliable functionality.
- Receives positive feedback from judges and the builder community.

Product Success

- Enables organisers to launch reward campaigns in under **2 minutes**.
- Allows participants to claim rewards in under **60 seconds**.
- Delivers fast, secure, and reliable NIM transactions.
- Encourages repeat engagement through passports, badges, and campaigns.

Ecosystem Success

- Increases real NIM transaction volume.
- Attracts new MiniPay users.
- Demonstrates a practical, reusable use case for Nimiq Mini Apps.
- Becomes a platform organisers return to for future events.

Part VI Deliverables

By the end of Part VI, the project has:

- A clear strategy aligned with every point of the **105-point judging rubric**.
- A structured marketing and **Build in Public** plan.
- Product analytics and success metrics.
- A compelling demo narrative.
- A comprehensive launch checklist.

- A phased roadmap extending beyond the competition.

This final section transforms RallyNIM from a technically complete Mini App into a compelling competition entry with a clear plan for adoption, visibility, and long-term ecosystem impact.